



**Course Title:** Software Requirements Engineering

**Course Code:** SWE-231

Assignment

On

Requirement Analysis

**Topic:** Library Management System

**Submitted to**

**Wadia Iqbal Chowdhury**

Lecturer

Department of Software Engineering

Metropolitan University, Sylhet

**Submitted by**

Uzma Yakub

**ID: 222-134-001**

**And**

Nadiya Rahman

**ID: 222-134-022**

**Batch:** 3rd

Department of Software Engineering

Metropolitan University, Sylhet



- Abstract

Requirement analysis stands as a pivotal phase in the software development life cycle, encompassing the identification, documentation, and comprehension of stakeholders' needs.

This assignment outlines the essential requirements to be considered during the development of a library management system. It outlines what the software should do, how it should behave, and what features it should include. It presents a comprehensive requirement analysis for the development of a library management system, encompassing software design, user interface, functional specialisation, and non-functional aspects.

It also includes use case diagrams to visualise user interactions and class diagrams to depict the system's structure.

In this assignment, we delve into the crucial process of requirement analysis as an integral part of developing a sophisticated library management system. We explore various methods, diagrams, and various aspects of requirement analysis that contribute to crafting a successful system that efficiently manages library resources while enhancing user experiences.

- Introduction

In an era of digital transformation, libraries are transitioning from traditional paper-based systems to digital platforms. A well-designed library management system is imperative for modern libraries to provide enhanced user experiences. It is a digital solution, designed to help libraries efficiently manage their operations, resources, and services. It is a crucial tool that is designed to streamline and automate various tasks performed by the users and the librarian.

The system aims to provide a seamless experience for users. It aims to provide users with the ability to borrow and return books and to create accounts and sign in to their account using different methods. It offers a user-friendly interface for users to discover books across genres, place orders, buy books, cancel orders, manage their accounts and interact with the cover page of the book containing author information.

It is designed to streamline the process of managing books, orders, and a library's book inventory. The librarian will have administrative privileges, enabling them to oversee tasks such as cataloguing books, managing loans, and efficiently handling user accounts.

- Software Design Requirements:

Software design requirements outline the overarching structure and technological foundation of the system. They encompass decisions on programming languages, frameworks, databases, and provide guidance on the overall architectural blueprint of the system.

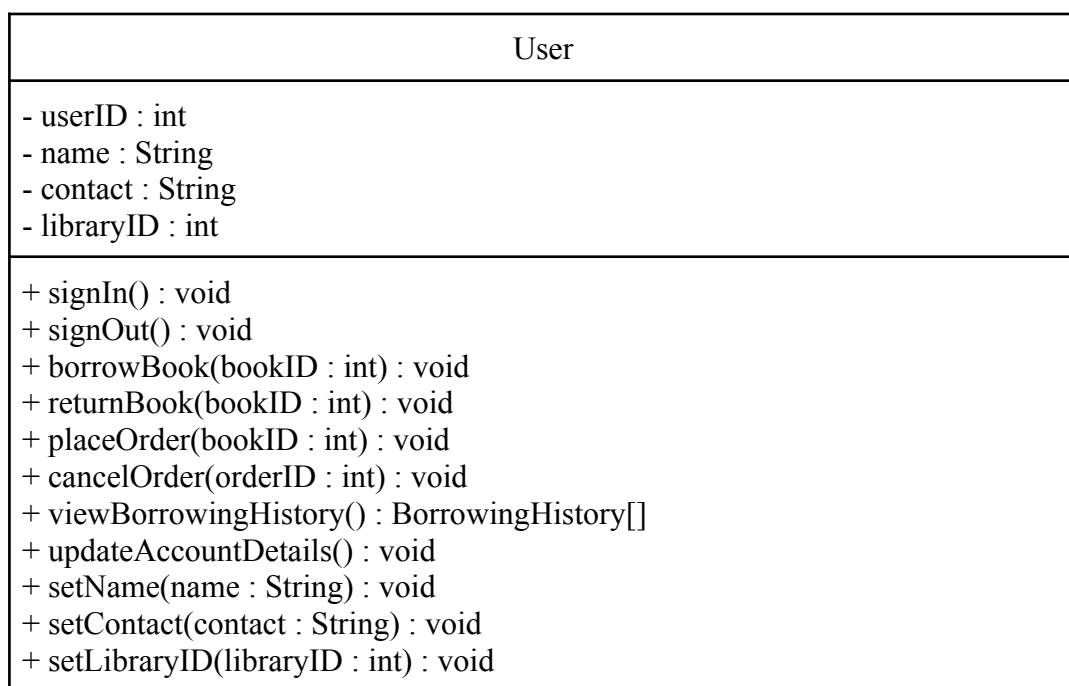
The software design requirements pertain to the overall structure and components of the library management system.

### Class Diagrams:

**Purpose of Class diagrams:** Class diagrams provide a structural view of the system's architecture. It provides an overview of the system's class structure. It depicts the relationships between classes(objects), their attributes and methods within the system. They aid in designing the software's structure, including the objects that interact and collaborate to achieve the desired functionalities.

In the context of a Library Management System, class diagrams represent classes like Users, Books, Orders, Genre, and Librarians(Library staff), showcasing their attributes and associations, highlighting the system's architecture and data flow. It aids in visualising the relationships between different components, such as 'Book,' 'User,' and 'Library Staff.'

Class diagrams play a pivotal role in shaping the software's underlying structure, outlining the entities and their interactions that facilitate the system's functionality.





| BorrowedBook                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>-bookID</li><li>- userID</li><li>- borrowDate</li><li>- returnDate : Date</li></ul>                                                                                                                   |
| <ul style="list-style-type: none"><li>+ getUserID()</li><li>+getBorrowDate()</li><li>+getReturnDate()</li><li>+setBookID(bookID)</li><li>+setUserID(userID)</li><li>+getBorrowDate(borrowDate)</li><li>+setReturnDate(returnDate)</li></ul> |



| Notification                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- notificationID : int</li><li>- userID : int</li><li>- message : String</li><li>- timestamp : Date</li></ul>                                                                                                                              |
| <ul style="list-style-type: none"><li>+ getNotificationID()</li><li>+ getUserID()</li><li>+ getMessage()</li><li>+ getTimestamp()</li><li>+setNotificationID(notificationID)</li><li>+ setUserID(userID)</li><li>+setMessage(message)</li><li>+setTimestamp(timestamp)</li></ul> |



|           |
|-----------|
| Librarian |
|-----------|

|                                                          |
|----------------------------------------------------------|
| - staffID : int<br>- name : String<br>- contact : String |
|----------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| + addBook(bookDetails : Book) : void<br>+ editBook(bookID : int, newDetails : Book) : void<br>+ deleteBook(bookID : int) : void<br>+ markBookAsCheckedOut(bookID : int) : void<br>+ markBookAsCheckedIn(bookID : int) : void<br>+ generateReports() : Report<br>+ setName(name : String) : void<br>+ setContact(contact : String) : void |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



| Book                                                                                                                                                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>-bookID : int<br/>- title : String<br/>- author : String<br/>- genre : String<br/>- availability : boolean</div>                                                                                                                                                                             |
| <div>+ getDetails() : String<br/>+ checkAvailability() : boolean<br/>+ updateAvailability(status : boolean) : void<br/>+ setTitle(title : String) : void<br/>+ setAuthor(author : String) : void<br/>+ setGenre(genre : String) : void<br/>+ setAvailability(availability : boolean) : void</div> |



| Genre                           |
|---------------------------------|
| <div>- genreID<br/>- name</div> |



| Order                                         |
|-----------------------------------------------|
| - orderID<br>- bookID<br>- userID<br>- status |
| + getStatus()<br>+ cancelOrder()              |



| BookPurchase                                                 |
|--------------------------------------------------------------|
| -paymentGateway                                              |
| +initiatePayment()<br>+processPayment()<br>+confirmPayment() |



| BorrowedBook                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -bookID: int<br>- userID: int<br>- borrowDate<br>- returnDate : Date                                                                                                                                                                          |
| + getUserID() : int<br>+ getBorrowDate() : Date<br>+ getReturnDate() : Date<br>+ setBookID(bookID : int) : void<br>+ setUserID(userID : int) : void<br>+ setBorrowDate(borrowDate : Date) : void<br>+ setReturnDate(returnDate : Date) : void |

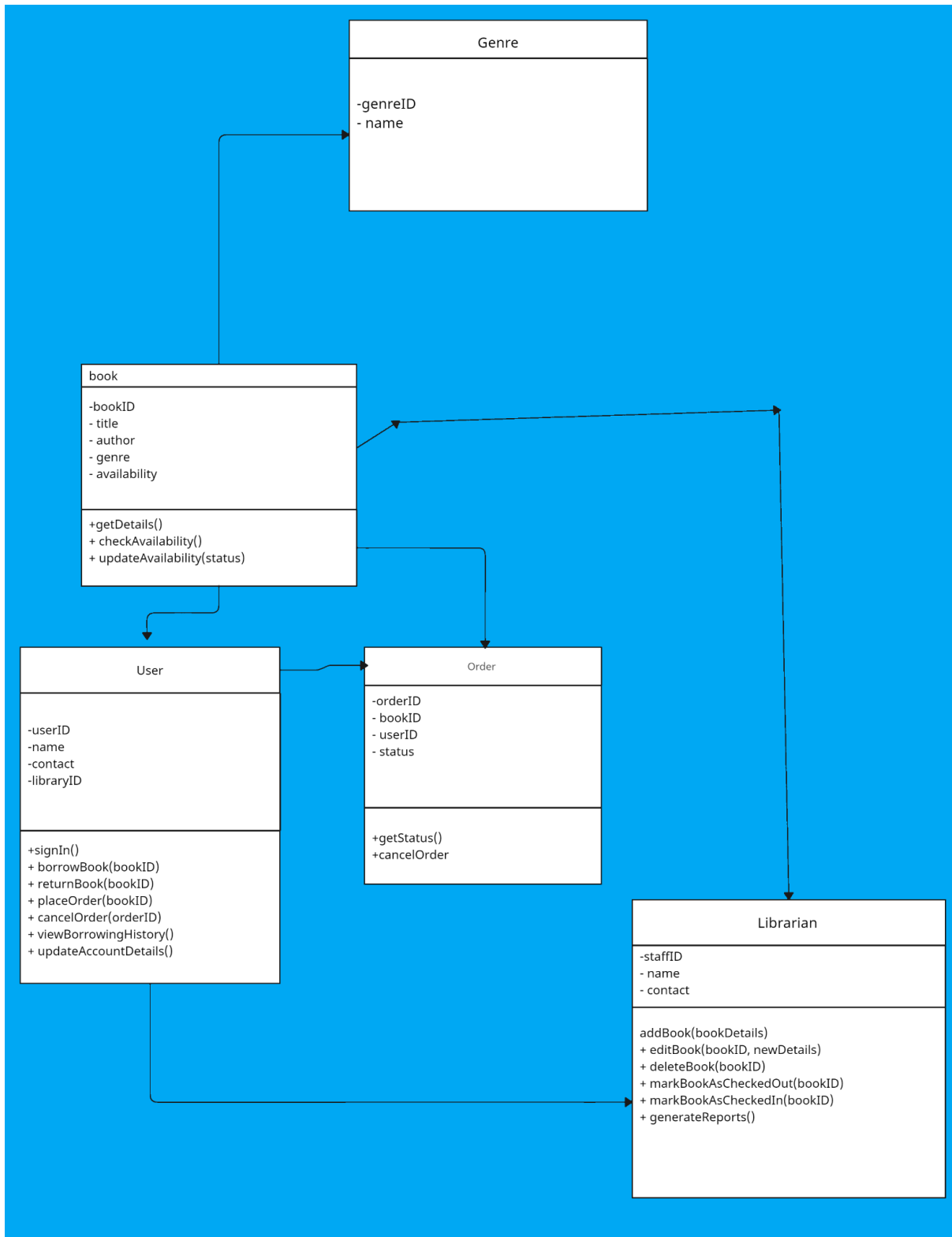


Fig: Class diagrams

- Functional requirements:



The functional requirements lay out the operational expectations of the library management system, dictating what the software should achieve. Within the context of this system, the functional requirements encompass a range of functionalities, including book registration, user authentication, book borrowing and returning, as well as efficient book search mechanisms.

#### 1) User Registration and Authentication:

Users can create accounts using Google account, phone number, or email address.

Account creation should require essential details like name, contact, and password

Verify the account via confirmation link, call or code.

Store user information such as name, contact details, and library ID.

Users can sign in using their registered credentials.

#### 2) Cover Page and Genre Display:

Display a list of available book genres.

Allow users to click on genres to view books and cover pages of books. Cover page should contain the author's name.

#### 3) Catalogue Management:

Librarians should be able to add, edit, and delete books from the catalogue. Librarians can mark books as checked in or checked out through the system.

Users should be able to search for books by title, author, or category. Patrons can check out books using their library accounts or Library cards, and the system will update the book's status accordingly.

Patrons can return books, and the system will update the book's availability status. Availability status of books should be displayed.

#### 4) Borrowing and Returning Books:

Users should be able to borrow books.

Users should be able to return books.

The system should enforce due dates and generate overdue notifications.

#### 5) Orders

Users can place orders for books.

Users can buy books.

Users can cancel orders.

#### 6) Reservation System:

Users should be able to reserve books that are currently borrowed.

Reserved books should be held for a specified period before being made available to others.

Patrons can place holds on books that are currently checked out by others.

The system will manage a hold queue, ensuring that holds are fulfilled in the order they were placed.

Librarians can mark holds as fulfilled when a book becomes available.

Patrons can place holds on unavailable books, and librarians can manage hold queues.

#### 7) User Profile Management:

Users should be able to view their borrowing history and account details.

Users should be able to update their account details including contact information.

Users should be able to sign out.

Librarians can view and user account information, including contact details and borrowing history.

#### 8) Notifications:

Users should receive notifications for due dates, overdue books, and reservation availability.

The system will send automated notifications to patrons for due dates, hold availability, and overdue fines.

Notifications can be sent via email or SMS based on user preferences.

#### 9) Admin Functions:

Admins should be able to manage user accounts and privileges.

Admins should be able to generate reports on book circulation, popular titles, etc.

- Functional Requirement Specialisation:

### 1) User Roles:

- Upon registration, users receive an email or SMS for account verification.
- Can browse, borrow, return, order, and purchase books.
- For patrons: Searching for books, checking availability, placing holds, renewing loans

### 2) Book Purchase:

Payment gateway integration for secure transactions.

### 3) Librarian Management:

Librarians can access a separate dashboard for inventory, loans, reports and book management.

Librarians can generate various reports, including:

- Book circulation report: Shows the number of check-outs and returns for each book.
- Overdue books report: Lists books that are overdue and the associated patrons.
- Popular genres report: Displays the most borrowed genres in a given timeframe.

Functional requirements describe the specific functionalities the system must support:

- Class diagrams: aid in representing the methods like 'borrowBook()' and 'returnBook()' within the 'User' class.

### ● Non-Functional Requirements:

#### 1) Performance:

The system should handle concurrent user requests efficiently, ensuring minimal response time even during peak usage.

Minimal system downtime: Book searches and transactions should have minimal response times.

Quick loading of book genres and cover pages.

#### 2) Reliability:

The system should maintain accurate records of book availability and user interactions.

Backup and recovery mechanisms should be in place to prevent data loss.

### 3) Security:

User authentication and authorization mechanisms should be robust. Protection against unauthorised access to sensitive user data.

User passwords and sensitive data should be securely stored and transmitted.

Secure connection for online transactions.

### 4) Scalability:

The system should be able to accommodate(handle) an increasing number of users, books, and transactions.

The software architecture should support the library's growth in terms of book collection and user base.

### 5) Availability:

- The system should be available for use during library operating hours.
- Scheduled maintenance and downtime should be communicated in advance.

### 6) Compatibility:

- The system should be compatible with different devices and browsers.

### 7) Data Integrity:

Data entered into the system should be accurate and consistent.

It should integrate with barcode scanners, online catalogues, and other library systems if needed.

Changes to data should be logged and auditable.

The system should integrate with barcode scanners and RFID technology for efficient book management.

Integration with online book catalogues and external APIs for book data enrichment should be considered.

### 8) Constraints and Assumptions

The system assumes that the library has existing book data that can be imported into the software.

It assumes that the library's network infrastructure supports the required communication for notifications.

9) Dependencies:

Identify any external systems (like databases, APIs) that the software will interact with.

10) Reviewing and Validating:

Reviewing the requirements document to ensure accuracy and alignment with expectations.

11) Documentating:

Compiling the requirements into a formal document that serves as a reference for the development team.

12) Iterating:

As development progresses, taking preparation for revisiting and updating requirements based on any new insights or changes.

- UI Requirements:

1) Create Account/Sign In:

- Clearly visible options for account creation and sign-in methods.

2) Genre Page:

- Intuitive display of genres for easy navigation.

3) Book Details:

- Cover page with prominent author name display.

4) Order Management:

- Clear option to cancel orders.

5) Librarian Interface:

- Distinct administrative interface for book management.

6) Usability

- Intuitive user interface for smooth navigation.
- The user interface should be user-friendly, catering to both librarians and patrons.
- Users need a search interface to find and request books, as well as view their account status.
- Users should be able to perform actions with minimal training.
- Support for multiple languages.

User Interface (UI) requirements focus on the design and user interaction aspects of the system:

Use Case Diagrams:

Purpose of use case diagrams:

Use case diagrams depict the functional aspects of the software and help in identifying the system's major functionalities and user roles. It serves as a compass, guiding the software development process by highlighting core functionalities and aiding in the identification of user roles. It facilitates an overall understanding of the system's behaviour

In the Library Management System, use case diagrams visually depicts (represents) how librarians and different users interact dynamically with the system's features to achieve specific goals, such as 'Librarian Adding a Book', 'User Searching for a Book', user registration, book browsing and order placement

| User | System                                                                                                     | Librarian |
|------|------------------------------------------------------------------------------------------------------------|-----------|
|      | Manage profile<br>Borrow book<br>Return book<br>Search book<br>Pay fine<br>Manage books<br>Generate report |           |

|      |
|------|
| User |
|------|

